Data Engineering Assignment 04

P. Vaishnavi
24NU1A4431
CSD - 2nd year

1. Write short notes on the following data storage systems
   a) single vs distributed storage
   b) Eventual vs strong consistency
   c) File, Block and object storage
   d) Cache and memory based storage.
   e) HDFS.

An

Single storage	Distributed Storage
• Data stored in one system or one server • Simple to manage and maintain. Ex:- A single database server like MySQL on one machine Adv:- Easy, Setup, low cost DisAdv:- Limited scalability, risk of failure (single point of failure).	• Data is stored across multiple systems or nodes • Used in large-scale applications. Ex:- Hadoop distributed File System (HDFS) Adv:- high scalability, fault tolerance. DisAdv:- Complex mangement

Strong Consistency	Eventual Consistency
• Data is immediately consistent after an update • All users see the same data at all same time Ex:- Banking System. Adv:- Accuracy DisAdv:- Slower Performance	• Data becomes consistent after some time. • Temporary differences may occur. Ex:- Social media field. Adv high availabity & performance DisAdv Temporary inconsistency

c) File, Block and Object Storage:-

File Storage:-

- Data stored as files in directories
- Ex: local file systems (NTFS)
- Easy to use and understand.

Block Storage:-

- Data stored in blocks with unique address
- Used in databases and virtual machines

Ex: Hard disks, SSD

Object Storage:-

- Data stored or objects with metadata and ID.
- used in cloud systems.

Ex:- Amazon S3

- Best for large unstructured data.

d) Cache and Memory-Based Storage:-

Cache Storage:-

- Temporary storage for frequently accessed data.
- Improves performance.
- Ex: CPU cache, Redis.

Memory-Based Storage:-

- Data stored in RAM
- Very fast access

Ex: In-memory databases.

Limitations: Volatile (data lost on power failure)

e) HDFS (Hadoop distributed file System):-

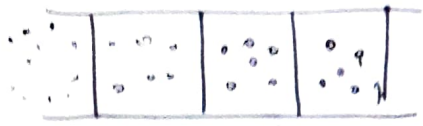
- A distributed file system used in big data.
- stores data large files across multiple machines.
- Works on master-slave architecture:-
  - NameNode (master)
  - DataNodes (slaves)

2. Discuss in detail key engineering features for the ingestion phase. Bounded vs unbounded, synchronous vs Asynchronous, serialization and deserialization, payload

Ans * Bounded vs Unbounded Ingestion:-

Bounded Ingestion:- The data intake is limited by predefined constraints such as fixed buffer size as time windows. The systems processes a known, finite amount of data, making resource allocation and error handling.

Unbounded Ingestion:- The system accepts data without strict limits, often handling streams of unknown lengths. It requires dynamic resource management and robust overflow handling to avoid crashes.



Bounded data

Unbounded data.

Synchronous Ingestion:- The data ingestion and processing happen sequentially in the same execution thread. It is similar simple to implement but can lead to blocking and reduced through put.

Asynchronous Ingestion:- Ingestion and processing occur independtly, often using separate threads or queues. It improves performance and responsiveness but adds complexity in managing concurrency and consistency.

* Serialization & Deserialization:-

Serialization :- converting complex data structures objects into a format (eg. JSON, Arro, or binary) suitable for storage or transmission during ingestion.

[object] → [serializer] → [serialized payload (JSON / Arro)]
→ [Transmission]
[Deserializer] → [Reconstructured Object]

De-serialization :- Reconstructing the original data structure from the serialized format for processing. Efficient serialization / deserialization reduces latency and ensures data integrity.

4) Payload Handling :-

- The payload refers to the actual data being ingested (eg.- messages, records)
  Engineering features involve optimizing payload size, format and validation to ensure efficient transmission and processing.
- payload design affects performance metrics like throughout, latency, and error stats during ingestion.

[Raw payload] → [validation] → [Transformation] → [Formatted payload]
↓
[size check] → [optimization] → [Ingestion]

3. Explain the concept and importance of "Change Data Capture (CDC)" in the content of data Eng Integration:

Ans Change Data Captures (CDC) is a data integration technique that identifies, capture and delivery the changes made to data sources in real time. Instead of moving entire data sets repeatedly, CDC tracks only the modifications - 'include', updates and 'deletes' - that occur in the same system.

Importance of CDC in data Engineering.

* Efficiency:- Reduces the volume of data transferred and processed, saving bandwidth and compute resources.
* Real-time analytics:- Enables timely synchronization of data pipelines, supporting up-to-data reporting and decision making.
* Data consistency:- Maintains integrity between source and target systems by applying only the actual changes.
* Performance:- Minimizes load on source databases because full extractions are avoided;

Typical CDC implementation Steps:-

* Identify change source - enable database logs or use triggers.
* Capture changes - extract log entries or triggers event that represent modification.
* Process & format - transform the raw change events into a consumable structure (eg. JSON)
* Deliver changes - Stream or batch the changes to the target system for application or storage.

Examples of CDC tools / approaches:-

- Log-based CDC (reading database transaction logs)
- Trigger based CDC (using database triggers to capture changes)
- Query-based CDC (periodic quires to detect differences).